

Guia de Tradução: Matemática para Programação

Esta tabela serve como um mapa mental para converter notações matemáticas de somatórios e produtórios em algoritmos que utilizam laços de repetição (*loops*).

Símbolo Matemático	Conceito no Código	Valor Inicial (Acumulador)
$\sum$ (Somatório)	<code>soma = soma + termo</code>	0 (Elemento Neutro da Soma)
$\prod$ (Produtório)	<code>prod = prod * termo</code>	1 (Elemento Neutro da Mult.)
$i = 1$ (Limite Inferior)	<code>i = k</code> (Exemplo: <code>i = 1</code>)	Início do laço
n (Limite Superior)	<code>i <= n</code> (Exemplo: <code>i <= 10</code>)	Condição de parada
$(-1)^i$ (Alternância)	<code>if (i % 2 == 0) ou multiplicador</code>	Trata sinais + e -
$i!$ (Fatorial)	Laço aninhado ou acumulador extra	1
$ \dots $ (Módulo)	<code>abs(valor)</code>	Função de Valor Absoluto
$\sqrt{x}$ (Raiz Quadrada)	<code>sqrt(x)</code>	Função de Radiciação
x^n (Potência)	<code>pow(x, n)</code>	Função de Potência

Tabela 1: Mapeamento de operadores para lógica de programação.

1 Exercícios de Programação: Laços de Repetição

Instruções

Para cada exercício abaixo, escreva um programa que calcule o resultado da fórmula utilizando apenas **laços de repetição** (como `for` ou `while`) e **variáveis acumuladoras**. Não é permitido o uso de vetores ou listas.

1. Progressão Aritmética (Soma Simples)

Calcule o valor de S dado o limite superior n :

$$S = \sum_{i=1}^n i$$

*Dica: Use uma variável **soma** inicializada em 0.*

2. Aproximação do Número de Euler (e)

O número de Euler pode ser aproximado pela soma dos inversos dos fatoriais. Calcule para um valor n :

$$e \approx \sum_{i=0}^n \frac{1}{i!}$$

Dica: O fatorial $i!$ também pode ser calculado acumulando multiplicações dentro do mesmo laço ou em um laço aninhado.

3. Série de Leibniz para π

Aproxime o valor de π usando a seguinte série alternada:

$$\pi \approx 4 \times \sum_{i=0}^n \frac{(-1)^i}{2i+1}$$

Dica: O termo $(-1)^i$ faz o sinal alternar entre positivo e negativo a cada iteração.

4. Cálculo de Potência via Produtório

Calcule o valor de x elevado a n (onde n é inteiro e positivo) sem usar funções prontas de potência:

$$P = \prod_{i=1}^n x$$

Dica: Diferente da soma, o acumulador de um produtório deve começar em 1.

5. Média Aritmética Dinâmica

Calcule a média de n números que serão lidos um a um do teclado (sem guardá-los em uma lista):

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Dica: Você só precisa de duas variáveis: uma para a `soma_total` e outra para o `valor_atual` lido.

6. Área do triângulo

Crie um programa que solicite as coordenadas (x_i, y_i) de 3 vértices de um triângulo e calcule sua área utilizando a fórmula do somatório:

$$A = \frac{1}{2} \left| \sum_{i=1}^3 (x_i y_{i+1} - x_{i+1} y_i) \right|$$

Onde, para o fechamento da figura, considere:

- Quando $i = 3$, o termo $i + 1$ refere-se ao primeiro vértice ($x_4 = x_1$ e $y_4 = y_1$).
- A é a área do triângulo.
- As barras verticais ($||$) indicam o valor absoluto (módulo), garantindo que a área não seja negativa.

2 Exercícios de Programação: Laços de Repetição - Parte 02

1. Implemente um programa que calcule a potência de um número base b elevado a um expoente inteiro e positivo n (b^n) **sem utilizar funções prontas** (como `pow`).

A lógica baseia-se no produtório:

$$P = \prod_{i=1}^n b$$

Dica: Utilize um laço de repetição que execute n vezes, multiplicando uma variável acumuladora (iniciada em 1) pela base b a cada iteração.

2. Escreva um programa que leia n valores reais (erros de previsão). Calcule e exiba a "Soma dos Erros Absolutos" (SEA), garantindo que valores negativos sejam tratados como positivos antes da soma.

A fórmula é dada por:

$$SEA = \sum_{i=1}^n |x_i|$$

3. Elabore um programa que calcule a evolução mensal de um investimento com aportes constantes. O usuário deve informar o capital inicial (P), a taxa de juros mensal (i) em decimal e o aporte mensal (PMT) durante n meses. O programa deve exibir o saldo ao final de cada mês.

A fórmula de atualização do saldo para cada mês t é:

$$M_t = M_{t-1} \cdot (1 + i) + PMT$$

Onde:

- M_t é o montante no mês atual.
- M_{t-1} é o saldo vindo do mês anterior (no mês 1, $M_0 = P$).
- i é a taxa de juros mensal.
- PMT é o valor depositado mensalmente.

*Dica: Use um laço **for** de 1 até n . Dentro do laço, atualize a variável **saldo** e imprima o valor atualizado em cada iteração.*

4. Implemente um programa que calcule a raiz quadrada aproximada de um número S utilizando o **Método de Newton-Raphson**, sem usar `sqrt()`.

O algoritmo deve repetir n vezes a seguinte atualização para uma estimativa x :

$$x_{novo} = \frac{1}{2} \left(x + \frac{S}{x} \right)$$

Dica: Comece com uma estimativa inicial $x = S/2$. Utilize um laço de repetição (ex: 10 iterações) para aplicar a fórmula acima. A cada volta, o valor de x fica mais próximo da raiz real.

5. Implemente um programa que imprima os números de 1 a 100, substituindo os múltiplos de 7 por palavras-chave específicas.

Por exemplo, substitua os múltiplos de 7 por "OpenAI", os múltiplos de 14 por "GPT", os múltiplos de 21 por "IA", e assim por diante.

Então, a saída do programa será:

1, 2, 3, 4, 5, 6, OpenAI, 8, 9, 10, 11, 12, 13, GPT, 15, 16, 17, 18, 19, 20, IA,
22, 23, 24, 25, 26, OpenAI, 27, 29, 30, 31, 32, 33, GPT, 35, 36, 37, 38, 39, 40, IA,
...

6. Implemente um programa que solicite ao usuário para inserir um número inteiro positivo n e, em seguida, imprima a soma das raízes quadradas e frações dos números de 1 até n .

Por exemplo:

- Se o usuário inserir o número 3, o programa deve calcular e imprimir a soma das raízes quadradas e frações de 1, 2 e 3:

$$\text{soma} = \sqrt{1} + \sqrt{\frac{1}{2}} + \sqrt{\frac{1}{3}}$$

- Se o usuário inserir o número 4, o programa deve calcular e imprimir a soma das raízes quadradas e frações de 1, 2, 3 e 4:

$$\text{soma} = \sqrt{1} + \sqrt{\frac{1}{2}} + \sqrt{\frac{1}{3}} + \sqrt{\frac{1}{4}}$$

Exemplo de execução no terminal para o número 3 inserido pelo usuário:

Insira um número inteiro positivo: 3

Soma das raízes quadradas e frações: 2.091089451179961

7. Implemente um programa que imprima os números de 1 a 100, substituindo os múltiplos de 3 por "Fizz", os múltiplos de 5 por "Buzz" e os múltiplos de ambos por "FizzBuzz".

Por exemplo:

- Se o número for múltiplo de 3, imprima "Fizz".
- Se o número for múltiplo de 5, imprima "Buzz".
- Se o número for múltiplo de ambos 3 e 5, imprima "FizzBuzz".

Então, a saída do programa será:

1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz, ...

8. Implemente um programa que solicite ao usuário para inserir um número inteiro positivo n e, em seguida, imprima a soma das potências e frações dos números de 1 até n .

Por exemplo:

- Se o usuário inserir o número 3, o programa deve calcular e imprimir a soma das potências e frações de 1, 2 e 3:

$$\text{soma} = \frac{1}{1} + \frac{1}{2^2} + \frac{1}{3^3}$$

- Se o usuário inserir o número 4, o programa deve calcular e imprimir a soma das potências e frações de 1, 2, 3 e 4:

$$\text{soma} = \frac{1}{1} + \frac{1}{2^2} + \frac{1}{3^3} + \frac{1}{4^4}$$

Exemplo de execução no terminal para o número 3 inserido pelo usuário:

```
Insira um número inteiro positivo: 3
Soma das potências e frações: 1.5555555555555556
```

9. Implemente um programa que solicite ao usuário para inserir um número inteiro positivo n e, em seguida, imprima a soma das potências dos números de 1 até n .

Por exemplo: Se o usuário inserir o número 3, o programa deve calcular e imprimir a soma das potências de 1, 2 e 3:

$$\text{soma} = 1^2 + 2^2 + 3^2$$

Se o usuário inserir o número 4, o programa deve calcular e imprimir a soma das potências de 1, 2, 3 e 4:

$$\text{soma} = 1^2 + 2^2 + 3^2 + 4^2$$

Exemplo de execução no terminal para o número 3 inserido pelo usuário:

```
Insira um número inteiro positivo: 3
Soma das potências: 14
```

10. Implemente um programa que imprima os números de 1 a 100, substituindo os números primos por "Prime" e os múltiplos de 7 por "GPT".

Por exemplo:

- Se o número for primo, imprima "Prime".
- Se o número for um múltiplo de 7, imprima "GPT".
- Caso contrário, imprima o próprio número.

Então, a saída do programa será:

```
1, 2, Prime, 4, 5, 6, GPT, Prime, 9, 10, Prime, 12, Prime, 14, Prime, ...
```

11. Implemente um programa que imprima os números de 1 a 100. Se a raiz quadrada de um número for um número inteiro, substitua-o por "Raiz"; se o número for um múltiplo de 5, substitua-o por "Múltiplos"; caso contrário, mantenha o número.

Por exemplo, se a raiz quadrada de um número for um número inteiro, substitua-o por "Raiz"; se o número for um múltiplo de 5, substitua-o por "Múltiplos"; caso contrário, mantenha o número.

A saída do programa será algo como:

1, 2, 3, 4, Raiz, 6, 7, 8, 9, Múltiplos, 11, 12, 13, 14, Raiz, 16, ...

12. Implemente um programa que solicite ao usuário inserir cinco anos e, para cada ano fornecido, verifique se é bissexto ou se é um ano de Copa do Mundo.

(a) Verifique se o ano é bissexto de acordo com as seguintes condições:

- Se o ano for divisível por 4 e não for divisível por 100, ou se for divisível por 400, então é bissexto.
- Caso contrário, não é bissexto.

(b) Verifique se o ano é um ano de Copa do Mundo. Os anos de Copa do Mundo ocorrem a cada quatro anos, começando em 1930, e são divisíveis por 4. No entanto, anos divisíveis por 100 são exceções, pois não são anos bissextos, então eles não são considerados anos de Copa do Mundo.

(c) Imprima o resultado para cada ano, indicando se é bissexto e se é um ano de Copa do Mundo.

13. Escreva um programa que determine se um número é um número de Armstrong ou não. Um número de Armstrong (também conhecido como número narcisista ou número pleno) é um número inteiro positivo que é igual à soma dos seus próprios dígitos elevados a uma potência igual ao número total de dígitos. Por exemplo, 153 é um número de Armstrong, pois $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$.

14. Implemente um programa que imprima os números de 1 a 100. Se um número for uma potência de 2, substitua-o por "Potência"; se a raiz quadrada de um número for um número inteiro, substitua-o por "Raiz"; caso contrário, mantenha o número.

Por exemplo, se um número for uma potência de 2, substitua-o por "Potência"; se a raiz quadrada de um número for um número inteiro, substitua-o por "Raiz"; caso contrário, mantenha o número.

A saída do programa será algo como:

1, Potência, 3, Raiz, 5, 6, 7, Potência, Raiz, 10, 11, Raiz, 13, 14, ...

15. Elabore um programa que leia um número e determine se é um número perfeito.

Um número perfeito é um número inteiro positivo que é igual à soma de seus divisores próprios positivos, excluindo ele mesmo. Por exemplo, o número 28 é perfeito porque a soma de seus divisores próprios (1, 2, 4, 7, 14) é igual a 28.

16. Desenvolva um programa que leia o código de um departamento e imprima seu nome correspondente, de acordo com uma tabela de mapeamento de códigos para nomes de departamentos. Solicite a entrada de cinco códigos de departamentos e suas respectivas faixas salariais. Ao final, apresente a soma total e a média salarial dos departamentos, excluindo aqueles definidos como 'Desconhecido'. Se um departamento for definido como 'Desconhecido', ele não deve ser incluído no cálculo da soma e da média salarial.

17. Implemente um programa que solicite ao usuário inserir um número inteiro positivo n . O programa deve calcular e imprimir a soma das frações dos números primos de 1 até n e a soma das raízes quadradas dos números não primos. Ao final, apresente a soma total.

Código	Departamento
1	Vendas
2	RH
3	TI
4	Marketing
Outros	Desconhecido

Por exemplo: - Se o usuário inserir o número 5, o programa deve calcular e imprimir a soma das frações dos números primos de 1 a 5 e a soma das raízes quadradas dos números não primos:

$$\text{soma} = \frac{1}{2} + \frac{1}{3} + \sqrt{4} + \frac{1}{5} + \sqrt{6}$$

- Se o usuário inserir o número 7, o programa deve calcular e imprimir a soma das frações dos números primos de 1 a 7 e a soma das raízes quadradas dos números não primos:

$$\text{soma} = \frac{1}{2} + \frac{1}{3} + \sqrt{4} + \frac{1}{5} + \frac{1}{7} + \sqrt{8}$$

Apresente também de forma individual, a soma para os números primos e não primos considerando as regras definidas anteriormente.

Exemplo de execução no terminal para o número 5 inserido pelo usuário:

```

Insira um número inteiro positivo: 5
Soma das frações dos números primos: 1.8666666666666667
Soma das raízes quadradas dos números não primos: 3.414213562373095
Soma total: 5.280880229039762

```

18. Desenvolva um programa que leia o código de barras EAN-13 e o preço de venda de um produto. Solicite a entrada de cinco códigos de barras e seus respectivos preços de venda. Ao final, apresente a soma total e a média dos preços de venda com o aumento de preço, excluindo aqueles definidos como 'Inválido'. Se um código for definido como 'Inválido', ele não deve ser incluído no cálculo da soma e dos preços. Aplique um aumento de preço de acordo com a categoria do produto, utilizando a seguinte tabela:

Código de Barras	Categoria	Aumento (%)
789 até 799	Bebidas	5%
800 até 899	Alimentos	10%
900 até 999	Limpeza	8%
Outros	Inválido	-

19. Elabore um programa que leia um número e determine se é um número deficiente.
- Um número deficiente é um número inteiro positivo cuja soma dos divisores próprios é menor que o próprio número. Por exemplo, o número 15 é deficiente porque a soma de seus divisores próprios (1, 3, 5) é igual a 9, que é menor que 15.
20. Implemente um programa que solicite ao usuário inserir um número inteiro positivo n . O programa deve calcular e imprimir a soma das frações dos números primos de 1 até n , elevando

cada número primo à potência de si mesmo, e a soma das raízes quadradas dos números não primos. Ao final, deve apresentar a soma total.

Por exemplo:

- Se o usuário inserir o número 5, o programa deve calcular e imprimir a soma das frações dos números primos de 1 a 5, elevando cada número primo à potência de si mesmo, e a soma das raízes quadradas dos números não primos:

$$\text{soma} = 2^2 + 3^3 + \sqrt{4} + 5^5 + \sqrt{6}$$

- Se o usuário inserir o número 7, o programa deve calcular e imprimir a soma das frações dos números primos de 1 a 7, elevando cada número primo à potência de si mesmo, e a soma das raízes quadradas dos números não primos:

$$\text{soma} = 2^2 + 3^3 + \sqrt{4} + 5^5 + 7^7 + \sqrt{8}$$

Exemplo de execução no terminal para o número 5 inserido pelo usuário:

```
Insira um número inteiro positivo: 5
Soma das frações dos números primos: 204.4
Soma das raízes quadradas dos números não primos: 3.44949
Soma total: 207.84949
```

21. Elabore um programa que leia um número e determine se é um número abundante.

Um número abundante é um número inteiro positivo cuja soma dos divisores próprios é maior que o próprio número. Por exemplo, o número 12 é abundante porque a soma de seus divisores próprios (1, 2, 3, 4, 6) é igual a 16, que é maior que 12.

22. Implemente um programa que solicite ao usuário inserir um número inteiro positivo n . O programa deve calcular e imprimir a soma das frações dos números pares de 1 até n e a soma das raízes cúbicas dos números ímpares. Ao final, apresente a soma total.

Por exemplo: - Se o usuário inserir o número 5, o programa deve calcular e imprimir a soma das frações dos números pares de 1 a 5 e a soma das raízes cúbicas dos números ímpares:

$$\text{soma} = \frac{1}{2} + \sqrt[3]{3} + \frac{1}{4} + \sqrt[3]{5}$$

- Se o usuário inserir o número 7, o programa deve calcular e imprimir a soma das frações dos números pares de 1 a 7 e a soma das raízes cúbicas dos números ímpares:

$$\text{soma} = \frac{1}{2} + \sqrt[3]{3} + \frac{1}{4} + \sqrt[3]{5} + \frac{1}{6} + \sqrt[3]{7}$$

Apresente também de forma individual, a soma para os números pares e ímpares considerando as regras definidas anteriormente.

Exemplo de saída no terminal para o número 5 inserido pelo usuário:

Insira um número inteiro positivo: 5
Soma das frações dos números pares: 0.75
Soma das raízes cúbicas dos números ímpares: 4.09808
Soma total: 4.84808

23. Implemente um programa que leia 3 notas de um conjunto de alunos e calcule a variância dessas notas. A variância é uma medida de dispersão que indica o quão distantes os valores estão da média.

- Solicite ao usuário que insira as 3 notas dos alunos.
- Calcule a média das notas.
- Para cada nota, calcule a diferença entre a nota e a média, eleve ao quadrado e some todas essas diferenças.
- Divida a soma das diferenças ao quadrado pelo número total de notas para obter a variância.

A fórmula para o cálculo da variância é dada por:

$$\text{Variância}(\sigma^2) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

Onde:

- σ^2 é a variância.
- N é o número total de notas.
- x_i são as notas individuais.
- $\bar{x}$ é a média das notas.

Exemplo:

Suponha que temos as seguintes notas de um conjunto de 3 alunos: 8, 7, 6.

Solução:

Calculando a variância:

$$\begin{aligned}\text{Média}(\bar{x}) &= \frac{8 + 7 + 6}{3} = 7 \\ \text{Variância}(\sigma^2) &= \frac{1}{3}[(8 - 7)^2 + (7 - 7)^2 + (6 - 7)^2] \\ &= \frac{1}{3}[1 + 0 + 1] \\ &= \frac{1}{3} \times 2 \approx 0.67\end{aligned}$$

Portanto, a variância das notas é aproximadamente 0.67.

24. **Cálculo da Série Alternada (Série de Gregory-Leibniz)**

Implemente um programa que calcule o valor aproximado de $\pi/4$ através da soma dos primeiros n termos da seguinte série:

$$S = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots$$

Regras do algoritmo:

- O usuário deve informar o número de termos n a serem somados.
- O programa **não deve** utilizar funções de potência (como `Math.pow`).
- **Desafio de Lógica:** Utilize uma variável `multiplicador` ou `sinal` que inverta seu valor (de 1 para -1 e vice-versa) a cada iteração do laço para tratar a alternância dos sinais.
- O denominador começa em 1 e aumenta de 2 em 2 a cada termo.

Dica: Ao final, multiplique o resultado total por 4 para ver o valor aproximado de π .

Tabela de Raciocínio (Teste de Mesa): A tabela abaixo ilustra como as variáveis devem se comportar internamente:

Termo (i)	Denominador	Sinal	Operação na Soma
0	1	1	$+(1/1)$
1	3	-1	$-(1/3)$
2	5	1	$+(1/5)$
3	7	-1	$-(1/7)$

25. O Jogo de Nim (O Último Perde)

Implemente um jogo interativo para dois jogadores. O jogo começa com um monte de 20 pedras. A cada rodada, um jogador pode remover 1, 2 ou 3 pedras. O objetivo é evitar retirar a última pedra, pois quem o fizer perde a partida.

Regras do algoritmo:

- O programa deve solicitar a quantidade de pedras a serem removidas pelo jogador da vez.
- **Validação:** O programa só deve aceitar os valores 1, 2 ou 3. Caso o usuário digite algo diferente, deve solicitar novamente até que um valor válido seja inserido.
- Após cada jogada, o programa deve exibir o saldo atual de pedras.
- O jogo termina quando o número de pedras chegar a 0.
- O programa deve alternar automaticamente entre "Jogador 1" e "Jogador 2".

Exemplo de execução no terminal:

```
Restam 20 pedras.
Jogador 1, retire 1, 2 ou 3: 3
Restam 17 pedras.
Jogador 2, retire 1, 2 ou 3: 2
...
Restam 1 pedra.
Jogador 1, retire 1, 2 ou 3: 1
FIM DE JOGO! Jogador 1 retirou a última pedra e PERDEU!
```

26. Cálculo de Média Ponderada com Validação de Pesos

Desenvolva um programa que calcule a média ponderada de três notas. O diferencial deste algoritmo é que ele deve garantir que a soma dos pesos seja exatamente 100% (ou 1.0) antes de realizar o cálculo.

Regras do algoritmo:

- O programa deve solicitar ao utilizador as 3 notas e os seus respetivos 3 pesos.
- **Validação de Integridade:** Antes de calcular a média, o programa deve verificar se a soma dos pesos fornecidos é igual a 100 (se usar valores inteiros) ou 1.0 (se usar valores decimais).
- Se a soma for diferente de 100%, o programa deve exibir uma mensagem de erro: "Erro: A soma dos pesos deve ser 100%" e não deve exibir a média.
- Caso os pesos estejam corretos, calcule a média utilizando a fórmula:

$$\text{Média} = \frac{(N_1 \cdot P_1) + (N_2 \cdot P_2) + (N_3 \cdot P_3)}{P_1 + P_2 + P_3}$$

Exemplo de execução (Erro):

Nota 1: 8.0, Peso 1: 50

Nota 2: 7.0, Peso 2: 20

Nota 3: 6.0, Peso 3: 20

Soma dos pesos: 90%

Erro: A soma dos pesos deve ser exatamente 100%.

Exemplo de execução (Sucesso):

Nota 1: 10.0, Peso 1: 40

Nota 2: 5.0, Peso 2: 30

Nota 3: 5.0, Peso 3: 30

Soma dos pesos: 100%

Média Final: 7.0

27. A Lenda de Sessa (O Tabuleiro de Xadrez)

Diz a lenda que o inventor do xadrez pediu como recompensa um grão de trigo pela primeira casa do tabuleiro, dois pela segunda, quatro pela terceira, e assim sucessivamente, dobrando a quantidade até a 64^a casa.

Implemente um programa que calcule:

- A quantidade de grãos em uma casa específica n (escolhida pelo usuário).
- O total de grãos acumulados desde a primeira casa até a casa n .

Regra: Não utilize `pow()`. Você deve usar um laço de repetição para calcular o dobro a cada passo.

28. Verificador de Números Primos

Escreva um programa que receba um número inteiro positivo e determine se ele é um número primo. Um número é primo se ele é divisível apenas por 1 e por ele mesmo.

Lógica sugerida:

- Peça ao usuário um número X .
- Use um laço para testar todos os divisores de 2 até $X - 1$.
- Se o resto da divisão de X por qualquer um desses números for 0, o número não é primo.

- **Desafio:** O programa deve imprimir "É primo" ou "Não é primo" apenas uma vez, ao final da execução.

*Dica: Use uma variável **contagem** para contar os divisores ou uma variável booleana **ehPrimo** que começa como verdadeira e muda para falsa se encontrar um divisor.*

29. A Conjectura de Collatz

Escolha qualquer número inteiro positivo n . Se n for par, divida-o por 2. Se n for ímpar, multiplique-o por 3 e some 1. Repita o processo com o novo valor até que o número chegue a 1.

Implemente um programa que:

- Receba o número inicial do usuário.
- Imprima cada passo da sequência.
- Ao final, diga quantos passos foram necessários para chegar ao número 1.

Exemplo para $n = 6$:

$6 \rightarrow 3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

Passos: 8.

3 Exercícios de Criptografia e Lógica de Dados

1. Cifra de César (Aritmética Modular)

Implemente um programa que realize a **Cifra de César**. O usuário deve inserir uma mensagem (apenas letras maiúsculas) e um valor de deslocamento k (chave). O programa deve transformar cada letra seguindo a fórmula:

$$E(x) = (x + k) \pmod{26}$$

Onde:

- x é a posição da letra no alfabeto (A=0, B=1, ..., Z=25).
- k é a chave de deslocamento (ex: se $k = 3$, 'A' vira 'D').
- O operador $\pmod{26}$ garante que o alfabeto seja circular (após 'Z' volta para 'A').

2. Cálculo de Dígito Verificador (Módulo 11)

Muitas vezes, dados são "protegidos" contra erros de digitação através de dígitos verificadores. Implemente um algoritmo que calcule o dígito de controle para uma sequência de 4 dígitos ($d_1d_2d_3d_4$) usando pesos decrescentes:

- Calcule a soma ponderada: $S = (d_1 \cdot 5) + (d_2 \cdot 4) + (d_3 \cdot 3) + (d_4 \cdot 2)$
- Calcule o resto da divisão de $S \cdot 10$ por 11.
- Se o resultado for 10, o dígito verificador é 0. Caso contrário, o resultado é o próprio dígito.

$$\text{Dígito} = (S \times 10) \pmod{11}$$

3. Inversão de Bits (Complemento de Um)

Escreva um programa que receba um número inteiro e realize a inversão de todos os seus bits (operador NOT). Embora pareça simples, essa é a base da representação negativa de números em sistemas computacionais.

- Entrada: x
- Saída: $\sim x$

Dica: Observe como o computador trata o bit de sinal ao realizar essa operação.